# Hinge for Roommates — System Design & Claude Build Plan

Nov 12, 2026 · @Manan

## Product overview & MVP scope

Build a native iOS + Android app with React Native (Expo) on a Supabase backend; it handles 1,000 users comfortably and scales to tens of thousands without a rewrite. The app matches people looking for a PG or flat roommate in the same city, using hard filters (city, gender rules, budget) plus a lifestyle compatibility score.

**Two kinds of users**

- **Seekers** — need a roommate *and* a place.
- **Hosts** — already have a room/flat and need someone to fill it. They show up in the normal feed *and* in the Rooms tab.

**MVP must-haves**

- Phone-number sign-up, onboarding with at least 3 photos, max budget, area, move-in date and lifestyle preferences
- Tinder-style gender controls: who you want to live with, and who your profile is shown to
- Hinge-style Discover feed: one full profile at a time, Like (optionally on a specific photo/answer, with a comment) or Cross
- Likes You, Matches + real-time chat, Rooms tab (replaces Standouts), Profile/edit
- Report, block, unmatch, delete account

**Not in v1:** payments/premium, video calls, ID verification, web version.

**On "replica of Hinge":** copy the *interaction patterns* (vertical profile card, liking a specific photo or prompt, tab layout), but use your own name, logo, colours and copy. App Store rule 4.1 (Copycats) rejects apps that mimic another app's branding, and Hinge's logo and assets are trademarked.

## Screens & navigation

Five bottom tabs, mirroring Hinge's layout with Standouts swapped for Rooms.

| Tab | What the user sees | Key actions |
|---|---|---|
| Discover | One profile at a time, scrolled vertically: photos interleaved with prompt answers and a lifestyle chip row (budget, area, veg/non-veg, smoking, drinking, move-in date), plus a match % | Heart a specific photo/prompt (optional comment), big X to cross, filter icon |
| Rooms | Hosts with a room available: room photos, rent, deposit, area, available-from, occupancy, plus host's mini profile | Filter by area/rent/date, "I'm interested" (counts as a like) |
| Likes You | Grid/list of people who liked you, with the photo/prompt they liked and their comment | Open profile → Like back (instant match) or Cross |
| Matches | Chat list sorted by latest message; new matches row on top | Chat, unmatch, report, block |
| Profile | Your profile preview, edit mode, preferences, settings | Edit photos/prompts/prefs, pause profile, delete account |

**Onboarding flow (one question per screen, Hinge-style):** phone OTP → name & date of birth (18+) → your gender → who you want to live with → who can see you → city & preferred areas → seeking a place or have a place → max budget & move-in date → lifestyle (food, smoking, drinking, sleep schedule, cleanliness, guests, pets, work/study) → mark dealbreakers → occupation → 3–6 photos → 3 prompts → (hosts) room listing.

**Tip:** the Mobbin connector you have access to includes real app screens; searching it for dating-app profile and onboarding flows gives Claude concrete UI references to describe.

## Tech stack

One TypeScript codebase for both phones, and a managed backend so you write almost no server code — the stack Claude writes most reliably.

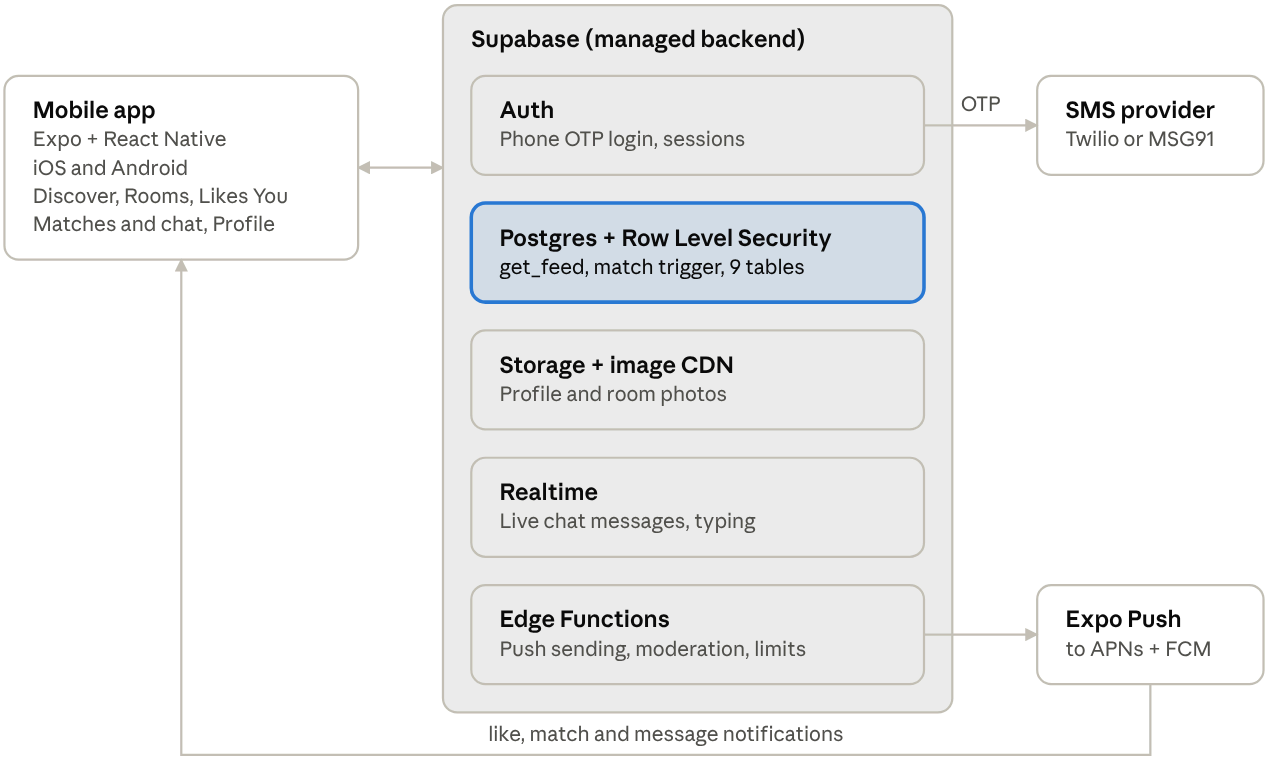
| Layer | Choice | Why |
|---|---|---|
| Mobile app | React Native + Expo (TypeScript), Expo Router | One codebase → iOS + Android; test instantly on your phone with Expo Go |
| UI | NativeWind (Tailwind for RN), Reanimated, expo-image | Smooth Hinge-like animations, cached images |
| State/data | TanStack Query + Zustand | Caching, pagination, optimistic likes |
| Auth | Supabase Auth — phone OTP (via Twilio/MSG91) | Indian users expect phone login; cuts fake accounts |
| Database | Supabase Postgres + Row Level Security | Relational data (likes, matches) fits SQL; RLS = security without a server |
| Photos | Supabase Storage + image transforms/CDN | Resize to ~1080px and serve thumbnails |
| Chat | Supabase Realtime (Postgres changes / broadcast) | Live messages without running a socket server |
| Server logic | Supabase Edge Functions + SQL functions | Feed ranking, match creation, push sending |
| Push | Expo Notifications (APNs + FCM) | New like / match / message alerts |
| Builds & release | EAS Build + EAS Submit | Cloud builds for Play Store and App Store without a Mac for Android |
| Monitoring | Sentry (crashes) + PostHog (analytics) | Free tiers cover 1k users |

Alternative: Firebase works too, but matching filters (budget ranges, multi-area, gender rules) are much easier in SQL than in Firestore.

## System architecture

The phone app never talks to a custom server: it calls Supabase directly, and Row Level Security in Postgres decides what each user may read or write.

**The app talks only to Supabase; Edge Functions send the pushes**




system architecture · app, Supabase, 2 outside services

Matching logic (get_feed and the match trigger) lives in Postgres, highlighted. Edge Functions handle anything that needs secret keys, such as sending pushes and moderating photos. Sentry and PostHog run inside the app for crash reports and analytics.

## Data model

Nine Postgres tables cover the whole MVP; every table has Row Level Security so users can only read what they're allowed to.

| Table | Key columns | Notes |
|---|---|---|
| profiles | id (= auth user), name, dob, gender, want_genders[], show_to_genders[], city, areas[], max_budget, move_in_date, seeking_type (`need_place` / `have_place`), occupation, bio, is_paused, last_active_at | want/show_to are arrays of `male`, `female`, `non_binary` |
| preferences | profile_id, food (veg/non-veg/egg), smoking, drinking, sleep (early/late), cleanliness (1–5), guests, pets, language[], dealbreakers[] | dealbreakers = fields that must match exactly |
| photos | id, profile_id, url, position, is_room_photo | Require ≥ 3 non-room photos before profile goes live |
| prompts | id, profile_id, question, answer, position | 3 per profile |
| listings | id, host_id, area, rent, deposit, available_from, occupancy, furnished, amenities[], room photos | Only for `have_place` users; feeds the Rooms tab |
| swipes | swiper_id, target_id, action (like/pass), liked_item (photo/prompt id), comment, created_at | Unique on (swiper, target) |
| matches | id, user_a, user_b, created_at, unmatched_at | Created automatically by a trigger on mutual like |
| messages | id, match_id, sender_id, body, created_at, read_at | Realtime-subscribed per match |
| reports_blocks | reporter_id, target_id, type (report/block), reason | Blocked users vanish from each other everywhere |

Indexes that matter: `profiles(city, gender, is_paused)`, `swipes(target_id, action)`, `messages(match_id, created_at)`, and a GIN index on `profiles.areas`. Also store `push_tokens(profile_id, token)`.

# Matching & feed algorithm

The feed runs in two stages: hard filters remove anyone incompatible, then a compatibility score orders the rest. It lives in one SQL function, `get_feed(user_id, limit 20, cursor)`, called by the app.

**Stage 1 — hard filters (all must pass)**

1. Same city, neither profile paused, both profiles complete (≥ 3 photos).
2. Gender, Tinder-style and two-way: show B to A only if `B.gender ∈ A.want_genders` **and** `A.gender ∈ B.show_to_genders`.
3. Budget overlaps: if B has a place, B's rent share ≤ A's max budget; if both are seekers, budgets within ±30% of each other.
4. Move-in dates within 30 days of each other.
5. No dealbreaker conflict (e.g. A marked "non-smoker only" and B smokes).
6. Not already swiped, not matched, not blocked or reported either way.

**Stage 2 — compatibility score (0–100, shown as "87% match")**

| Factor | Weight | How it's scored |
|---|---|---|
| Area overlap | 25 | Shared preferred areas / neighbouring areas |
| Food | 15 | Veg + veg = full; veg + non-veg = low (common PG friction) |
| Smoking & drinking | 20 | Same habit or both "okay with it" |
| Budget closeness | 15 | Closer budgets score higher |
| Sleep & cleanliness | 15 | Same schedule; cleanliness within 1 point |
| Guests, pets, language | 10 | Matching answers |

**Final ranking** = score, plus a small boost for profiles active in the last 3 days and for people who already liked you (so likes get seen). Hosts with an open listing get a mild boost for seekers who `need_place`.

**Likes limit:** cap at ~25 likes/day per user to stop spam-liking; tune later.

## Likes, matches, chat & notifications

A match is created by the database itself the moment two people have liked each other, so it can never be missed or duplicated.

1. A likes B (optionally on a photo/prompt, with a comment) → row in `swipes`.
2. A Postgres trigger checks: has B already liked A? If yes → insert into `matches`, and both get a "It's a match" push.
3. If no → B gets a "Someone liked you" push, and A appears in B's Likes You tab (with the liked item and comment).
4. Cross = `pass` row; that profile won't reappear (optionally recycle passes after 30 days if the city runs out of people).
5. Chat opens only for matches. Messages are inserted into `messages`; both phones subscribe to that match's channel via Supabase Realtime, so messages arrive instantly. RLS ensures only the two matched users can read or send.
6. A database webhook → Edge Function sends Expo push for new messages when the recipient isn't in the chat.
7. Unmatch sets `unmatched_at`, hides the chat for both; block also removes them from each other's feeds forever.

Extras worth adding early: read receipts (`read_at`), typing indicator (Realtime broadcast), and a "Your turn" badge on chats awaiting your reply — a Hinge touch that keeps conversations moving.

## Scaling to 1k+ users & cost

1,000 users is small for this stack; the same design holds to roughly 50k users with only plan upgrades. What actually matters at this size:

- **Photos are the biggest cost.** Compress on upload (≤ 1080px, ~200 KB WebP), serve thumbnails in lists. 1k users × 6 photos ≈ 1.2 GB.
- **Paginate everything** — feed in batches of 20, chats load the last 30 messages, infinite scroll upward.
- **Do filtering in SQL**, never download all profiles to the phone.
- **Indexes** listed in the data model; run `EXPLAIN` on `get_feed` once you have seed data.
- **Rate limits** on likes, messages and reports (Edge Function or SQL check).
- **Seed 1,000 fake profiles** with a script to load-test the feed before launch.

| Item | Approx. cost | Notes |
|---|---|---|
| Supabase | Free to start; Pro ~US$25/month | Move to Pro before launch (no project pausing, daily backups) — check current pricing |
| SMS OTP (Twilio/MSG91) | ~₹0.2–1 per OTP | MSG91 is usually cheaper in India; needs DLT registration |
| Google Play developer account | US$25 one-time | |
| Apple developer program | US$99/year | Needed for App Store |
| Expo EAS | Free tier enough for a few builds/month | |
| Sentry, PostHog | Free tiers | |
| Claude (Pro/Max plan) | Your subscription | Claude Code is included on paid plans |

Figures are approximate from general knowledge; confirm on each provider's pricing page.

## Safety, privacy & trust

People will meet strangers and possibly share a home with them, so trust features are part of the MVP, not extras.

- **Verified phone numbers** for every account; 18+ check at sign-up.
- **No exact addresses** on listings before a match — show area/locality only.
- **Report & block** on every profile and chat; reported users auto-hidden after N reports pending review.
- **Photo moderation** — run uploads through a moderation API (or manual review at launch) to block nudity and non-person photos.
- **Selfie verification badge** (v2) to fight catfishing.
- **Safety tips** card before the first chat: meet in public, visit the flat together, never pay deposits before seeing the place.
- **Privacy law:** India's DPDP Act 2023 applies — publish a privacy policy, collect consent, let users download/delete their data.

- **In-app account deletion** is mandatory on both App Store and Google Play.
- **Secrets:** Supabase service-role key only in Edge Functions, never in the app; RLS on every table.

## Build plan: the exact Claude prompts

A real app takes about 10 focused prompts, not 2–3 — one per feature, each followed by testing on your phone. Trying to get everything in one prompt produces a big app that half-works and is hard to fix. Expect 2–4 weeks part-time.

### Step 0 — setup (once, ~1 hour)

- Install Node.js LTS, Git, and the Expo Go app on your phone.
- Install **Claude Code** (the coding version of Claude that reads and writes files in your project) and log in with your Claude account.
- Create a free Supabase project and a GitHub repo.
- Make an empty folder, open Claude Code in it, then send the prompts below one at a time.

### Prompt 1 — the spec file (most important prompt)

```
Create a CLAUDE.md file for a React Native app I'm building. It is a "Hinge for roommates" app for finding PG/flat roommates in Indian cities. Put this whole spec in it: [paste sections 1–7 of this doc]. Tech stack: Expo (TypeScript, Expo Router), NativeWind, TanStack Query, Zustand, Supabase (Auth with phone OTP, Postgres with RLS, Storage, Realtime, Edge Functions), Expo Notifications. Add coding rules: small reusable components, typed Supabase client, no secrets in the app, every table has RLS. Do not write app code yet.
```

### Prompt 2 — scaffold + design system

```
Read CLAUDE.md. Use plan mode: propose the folder structure first, wait for my OK. Then scaffold the Expo app with Expo Router and 5 bottom tabs (Discover, Rooms, Likes You, Matches, Profile) with placeholder screens. Create a design system inspired by Hinge's clean, minimal style but with our own brand: name "[YourAppName]", colours, serif headings + sans body, rounded cards, large photos. Make it run in Expo Go.
```

### Prompt 3 — database

```
Create Supabase SQL migrations for all tables in CLAUDE.md with indexes and Row Level Security policies. Add the trigger that creates a match on mutual like, and the get_feed SQL function with the hard filters and compatibility score exactly as specified. Write a seed script that creates 200 realistic fake profiles across Bengaluru, Pune and Mumbai. Explain how I apply the migrations.
```

### Prompt 4 — auth + onboarding

```
Build phone-OTP login with Supabase Auth and the Hinge-style onboarding flow from CLAUDE.md, one question per screen with a progress bar. Include gender, want_genders and show_to_genders as multi-select; seeking vs have-a-place; budget slider; lifestyle and dealbreakers; photo picker requiring at least 3 photos (compress to 1080px WebP before upload); 3 prompts. Users can't reach the tabs until the profile is complete.
```

### Prompt 5 — profile card + Discover

```
Build the ProfileCard component: vertical scroll of photos interleaved with prompts, a lifestyle chip row, and the match %. Each photo/prompt has a heart button that opens a 'Like with comment' sheet. Add a floating X to cross. Build the Discover tab using get_feed with pagination and optimistic updates. Add a filter sheet (areas, budget, move-in).
```

### Prompt 6 — Likes You + matches

```
Build the Likes You tab showing who liked me with the item they liked and their comment; tapping opens their full ProfileCard with Like back / Cross. Liking back must create the match via the trigger and show an 'It's a match' screen.
```

### Prompt 7 — chat

```
Build the Matches tab: new-matches row on top, conversations list below sorted by latest message with a 'Your turn' badge. Build the chat screen with Supabase Realtime, read receipts, pagination of older messages, and unmatch/report/block in the menu.
```

### Prompt 8 — Rooms tab

```
Build the Rooms tab and listing creation for have_place users: room photos,
rent, deposit, area, available_from, occupancy, amenities. Browse with
filters; 'I'm interested' counts as a like on the host. Never show exact
address before a match.
```

**Prompt 9 — notifications, safety, settings**

```
Add Expo push notifications for new like, new match and new message via a
Supabase database webhook + Edge Function. Add report/block flows, daily like
limit of 25, pause profile, edit profile, and in-app account deletion that
removes all user data and photos.
```

**Prompt 10 — hardening + release**

```
Review the whole codebase for bugs, security holes (check every RLS policy),
and performance issues with 1,000+ users. Add Sentry and PostHog. Write tests
for get_feed gender/budget filters and the match trigger. Then set up EAS
Build and walk me step by step through publishing to Google Play internal
testing and TestFlight.
```

## How to use Claude well

The biggest difference between a working app and a broken one is the loop: **plan → build one feature → test on your phone → commit → next**.

- **Use Claude Code, not chat,** for building. It sees every file, runs commands and fixes its own errors. Use chat for thinking through design questions.
- **CLAUDE.md is Claude's memory of the project.** Keep it updated whenever you change a decision ("we added a pets filter") so every new session starts informed.
- **Plan before code.** Start feature prompts with "plan first, don't code until I approve". Correct the plan; it's 10× cheaper than correcting code.
- **One feature per session.** Run `/clear` between features so old context doesn't confuse it.
- **Test after every prompt** in Expo Go. Report bugs precisely: screen, what you did, what happened, and the exact error text or a screenshot.
- **Commit to Git after each working step** ("commit this with a clear message"). If Claude breaks something, you can roll back.
- **Ask for reviews.** "Review the RLS policies as a security engineer" or "what breaks with 10,000 users?" catches problems before users do.

- **Connect Supabase to Claude Code** (Supabase has an MCP server) so Claude can apply migrations and inspect tables directly.
- **Don't paste secret keys into prompts;** put them in `.env` and tell Claude the variable names.
- **Show, don't describe, the UI.** Screenshots of the screen you want (or a Mobbin reference) give far better results than adjectives.

## Launch checklist

Launch in one city first (e.g. Bengaluru) so the feed isn't empty — a matching app with 1k users across 10 cities feels dead.

- [ ] Pick a name, logo and colours that are your own
- [ ] Supabase on Pro plan, backups on, RLS checked on every table
- [ ] SMS provider set up (DLT registration for India)
- [ ] Privacy policy + terms hosted on a simple webpage
- [ ] In-app account deletion works end to end
- [ ] Seeded/test with 20–30 real friends; fix top bugs
- [ ] Sentry + PostHog live
- [ ] Google Play internal testing → closed testing (Play requires a testing period for new personal accounts)
- [ ] TestFlight beta → App Store review
- [ ] Seed supply before launch: college groups, PG WhatsApp/Telegram groups, office communities in one city